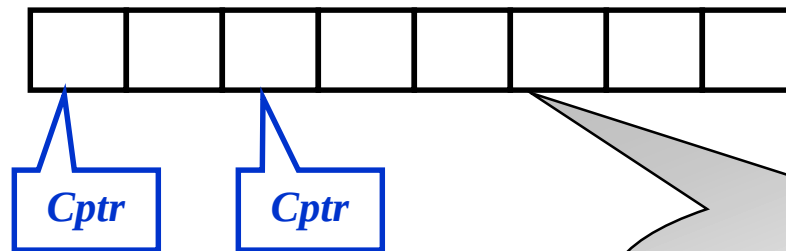
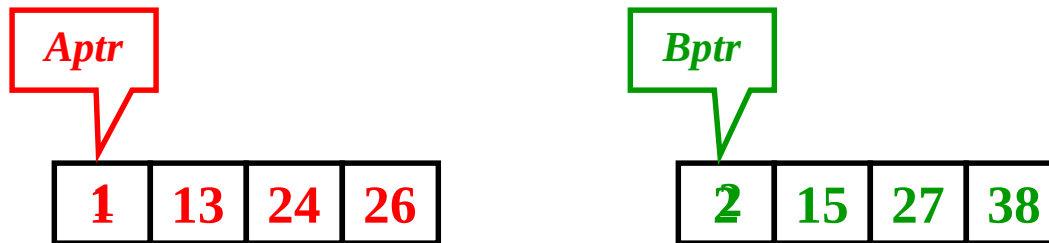


❖ 归并排序

- 将两个已排序的子序列合并成一个有序序列。



二路归并。
也可以使用多路归并。

- $T(N) = O(N)$, N 是元素个数。
- $S(N) = O(N)$ 。
- 方法是稳定的。

```
void Merge( ElementType A[], ElementType TmpA[],
int Left, int Mid, int Right )
{ /* 将有序的A[Left]~A[Mid-1]和A[Mid]~A[Right]归并成一个有序序列 */
    int Tp, LeftEnd, i;

    Tp = Left;          /* 有序序列的起始位置 */
    LeftEnd = Mid - 1; /* 左边序列终止的位置 */
    while ( ( Left < LeftEnd ) && ( Mid < Right ) )
        if ( A[Left] <= A[Mid] )
            TmpA[Tp++] = A[Left++]; /* 将左边元素复制到TmpA */
        else
            TmpA[Tp++] = A[Mid++]; /* 将右边元素复制到TmpA */
    while ( Left <= LeftEnd ) /* 如果左边有元素剩下而右边已经结束 */
        TmpA[Tp++] = A[Left++]; /* 将左边剩余元素复制到TmpA */
    while ( Mid <= Right ) /* 如果右边有元素剩下而左边已经结束 */
        TmpA[Tp++] = A[Mid++]; /* 将右边剩余元素复制到TmpA */
    for ( i=Right-Left; i>=0; i--, Right-- )
        A[Right] = TmpA[Right]; /* 将有序的TmpA[]复制回A[] */
}
```

```

void MSort( ElementType A[ ], ElementType TmpA [ ],
            int Left, int Right )
{
    int Center;          /* 递归地将A[Left]~A[Right]排序 */
    if ( Left < Right ) { /* 如果还有元素要排序 */
        Center = ( Left + Right ) / 2;
        MSort( A, TmpA, Left, Center ); /*递归排左半边  $T(N/2)$  */
        MSort( A, TmpA, Center + 1, Right ); /*递归排右半边 $T(N/2)$  */
        Merge( A, TmpA, Left, Center + 1, Right ); /*归并,  $O(N)$  */
    }
}

```

```

void Mergesort( ElementType A, int n )
{
    ElementType TmpArray[ MAX_SIZE ];
    MSort( A, TmpArray, 0, n-1 );
    free( TmpArray );
}

```

如果 TmpA 定义成Merge的局部变量,
则每次调用Merge都需要开辟不同的空间, 那么 $S(N) = O(N \log N)$

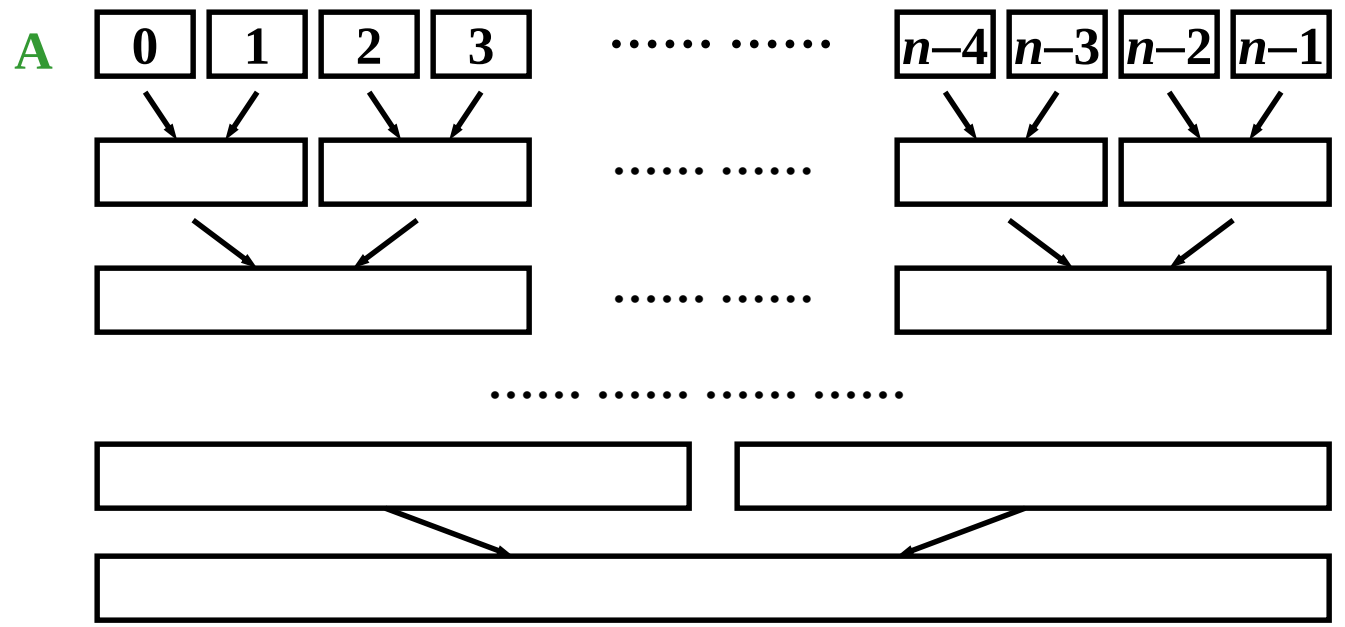
➤ 时间复杂性分析:

$T(1) = 1$

$$\begin{aligned} T(N) &= 2T(N/2) + O(N) \\ &= 2^k T(N/2^k) + k * O(N) \\ &= N * T(1) + \log N * O(N) \\ &= O(N + N \log N) \end{aligned}$$

注意: 归并排序需要线性的额外存储, 并且经常复制临时数组导致效率降低。它很少用于内部排序, 然而在外部分排序中非常常用。

➤ 非递归的算法思想:

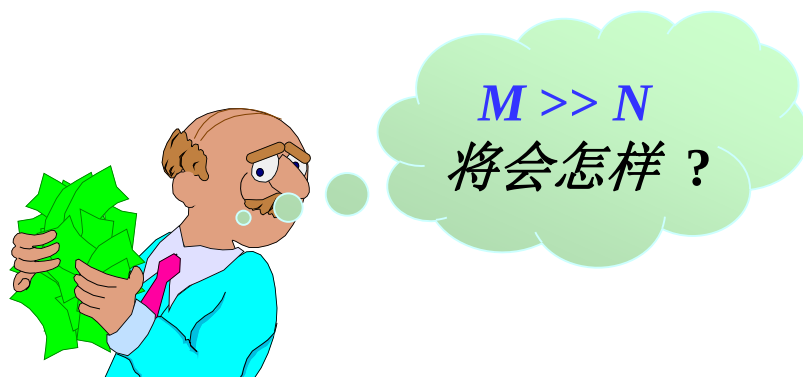
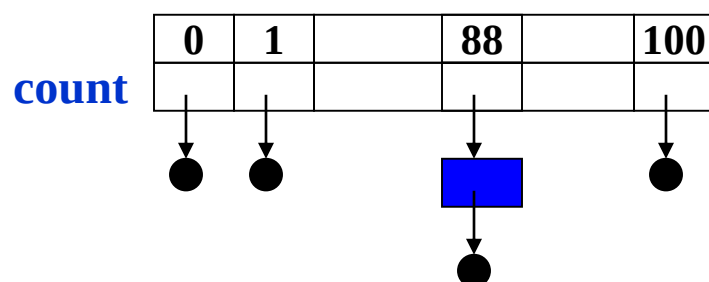


❖ 基数排序

➤ “**基数排序** (Radix Sort)” 是 “**桶排序** (Bucket Sort)” 的推广。

👉 桶排序

【例】假如有 N 个学生，每个人的成绩分布在 0 到 100 (令 $M = 101$ 种可能的成绩)。如何在**线性的时间**内按照成绩给他们**排序**？



算法思想：

```
{
    初始化链表头数组 count[ ];
    while (读一个学生的成绩)
        把它插入链表 count[stdnt.grade];
    for (i=0; i<M; i++) {
        if (count[i])
            输出链表count[i];
    }
}
```

$T(N, M) = O(M+N)$

☞ **基数排序**：一般用于多关键字的排序

【例】一叠牌的排序要基于**两个关键字**。

第一关键字[花色] ♣ < ♦ < ♥ < ♠

第一关键字[面值] 2 < 3 < 4 < 5 < 6 < 7 < 8 < 9 < 10 < J < Q < K < A

排序结果应该是： ♣2 ... ♣A ♦2 ... ♦A ♥2 ... ♥A ♠2 ... ♠A

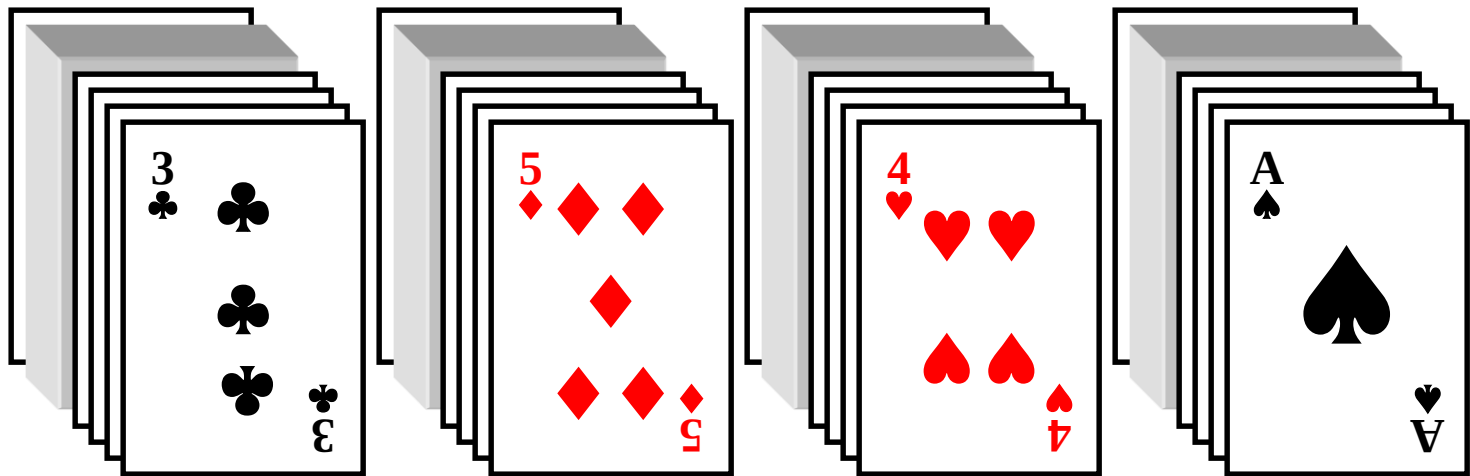
☞ 基数排序的方法一般采用“**主位优先法**” (**Most Significant Digit First, MSD**) 或者“**次位优先法**” (**Least Significant Digit First, LSD**)

“分配-收集”法

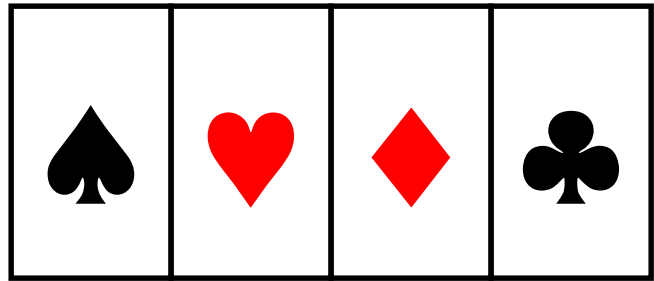
相当于“分治法”

☞ 主位优先法(MSD) 排序

① 先排花色: 比如, 建立4个花色的“桶”

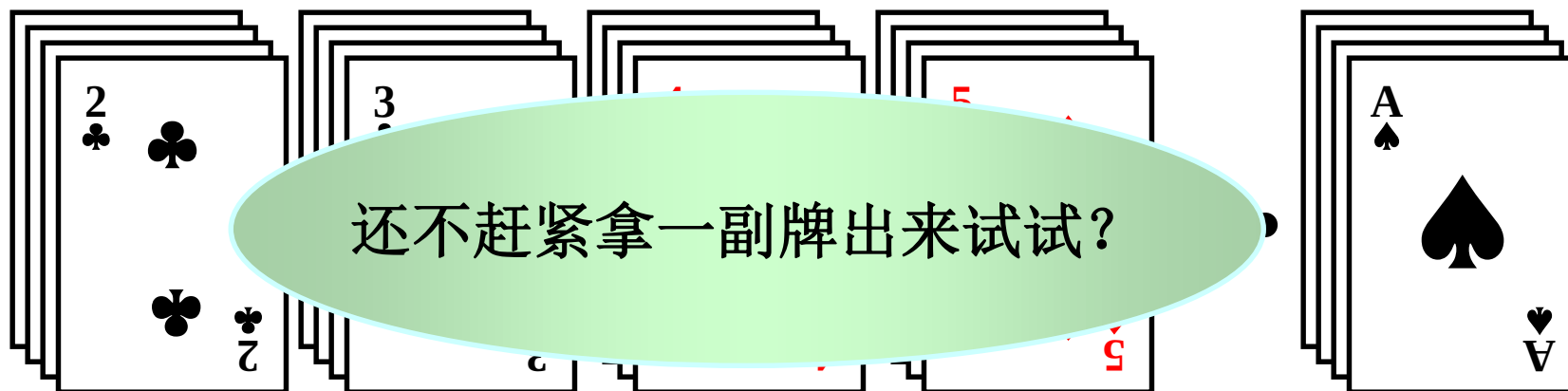


② 再排面值: 每个桶分别独立排序(可以采用以前介绍的任何排序方法)



👉 次位优先法(LSD) 排序

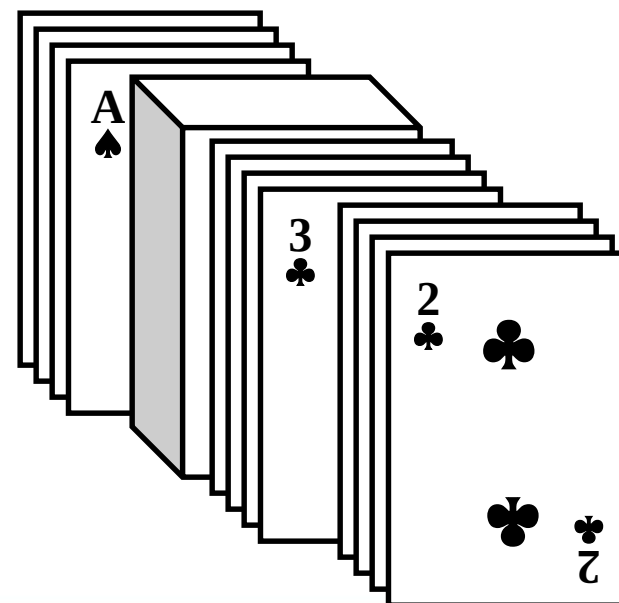
① **先排面值**: 建立13个面值的“桶”，把牌放入相应的桶中（这叫“**分配**”）；



② 依次“**收集**”它们成为一叠牌；

③ **再排花色**: 建立4个桶进行再“**分配**”；

④ 再次“**收集**”它们成为一叠牌即告完成。



☞ 单关键字的基数排序

【例】 给定 $N = 10$ 个整数，范围介于 0 到 999 ($M = 1000$)
可以在**线性的时间**内把它们排序？

按“**主位优先法**”排序将会怎样？

输入：64, 8, 216, 512, 27, 729, 0, 1, 343, 125

桶：	0	1	2	3	4	5	6	7	8	9
轮次 1	0	1	512	343	64	125	216	27	8	729
轮次 2	0	512	125		343		64			
	1	216	27							
	8		729							
轮次 3	0	125	216	343		512		729		
	1									
	8									
	27									
	64									

用**链式**储存更便于**分配与收集**？

输出：0, 1, 8, 27, 64, 125, 216, 343, 512, 729

时间复杂性：
 $T=O(D(N+R))$
其中**D**是轮次，
N 是元素个数，
R 是桶的数量。

空间复杂性：
 $S=O(N+R)$ ，
N 是元素个数，
R 是桶的数量。

稳定性： **稳定**

```

typedef struct Node *PtrToNode;
typedef PtrToNode List;
struct Node {
    int Key[MaxDigit]; /* MaxDigit是关键字个数,Key可以定义成 char类型数组 */
    PtrToNode Next;
};
List RadixSort( List A ) /* 基数排序 */
{
    List Bucket[Radix]; /* 建立Radix个桶 */
    List Rear[Radix]; /* 需要记录每个桶链表的尾元素位置 */
    int i, j, Digit;
    for ( i = MaxDigit-1; i >= 0; i-- ){ /* 从最次位关键字开始 */
        for ( j=0; j<Radix; j++ )
            Bucket[j] = Rear[j] = NULL; /* 初始化 */
        while (A) { /* 将关键字逐一分配到桶 */
            Digit = A->Key[i]; /* 取出当前关键字位 */
            if ( !Bucket[Digit] ) /* 若对应的桶是空的 */
                Bucket[Digit] = A; /* 放入空桶 */
            else Rear[Digit]->Next = A; /* 否则放入桶尾 */
            Rear[Digit] = A; /* 更新尾指针 */
            A = A->Next;
        } /* 结束while分配过程*/
        for ( j=Radix-1; j>=0; j-- ){ /* 将所有桶中元素收集串联 */
            if ( Bucket[j] ) {
                Rear[j]->Next = A;
                A = Bucket[j];
            }
        } /* 结束for收集过程 */
    }
    return A;
}

```

❖ 外部排序*(External Sort)

👉 内部排序和外部排序

👉 分两个步实施：(a) 分段内排序 (b) 归并（二路或多路）

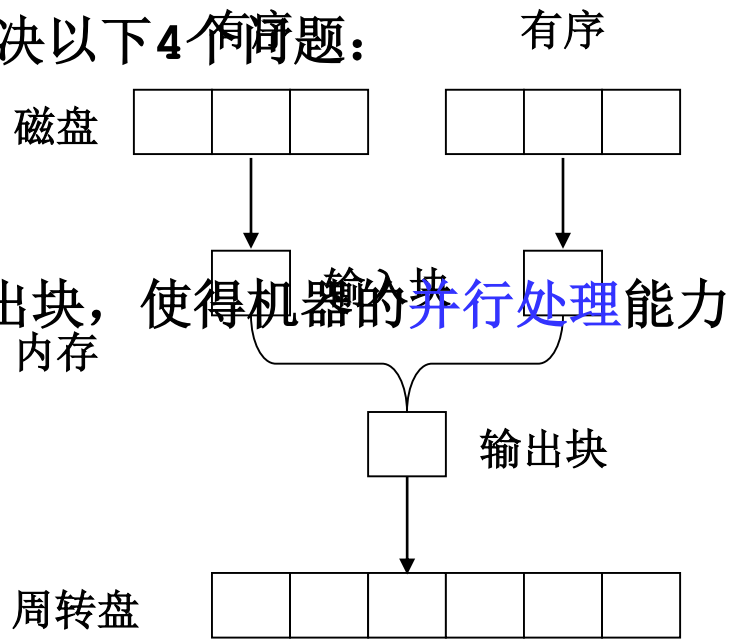
👉 要提高外排的效率，关键要解决以下4个问题：

✎ 如何减少归并轮数；

✎ 如何有效安排内存中的输入、输出块，被最大限度地利用；

✎ 如何有效生成归并段；

✎ 如何将归并段进行有效归并。



初始序列基本有序时，效率高

排序方法	平均时间复杂度	最坏时间复杂度	额外空间复杂度	稳定性
简单选择排序	$O(N^2)$	$O(N^2)$	$O(1)$	不稳定
直接插入排序	$O(N^2)$	$O(N^2)$	$O(1)$	稳定
冒泡排序	$O(N^2)$	$O(N^2)$	$O(1)$	稳定
希尔排序	$O(N^d) (1 < d < 1.5)$	$O(N^2)$	$O(1)$	不稳定
堆排序	$O(N \log_2 N)$	$O(N \log_2 N)$	$O(1)$	不稳定
快速排序	$O(N \log_2 N)$	$O(N^2)$	$O(\log_2 N)$	不稳定
归并排序	$O(N \log_2 N)$	$O(N \log_2 N)$	$O(N)$	稳定
基数排序	$O(D(N+R))$	$O(D(N+R))$	$O(N+R)$	稳定

尽管最坏情况不好，但实际的平均时间效率上，快速排序是最佳的排序方法

关键字比较

基于关键字与地址的映射